CS-2001
Data Structures
Fall 2023

Stack ADT Applications

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

Topics to be covered

Previous Lecture

- Introduction to stack ADT
  - Applications
  - Array-based implementation
  - Linked list based Implementation

Today

- Postfix Notation
- Prefix Notation
- Postfix Expression evaluation and use of stack

Polish Notation and Use of Stack

Algebraic Expressions

- An algebraic expression is combination of operands and operators
- Operand is a quantity that is operated on
- Operator is a symbol that signifies a mathematical or logical operation

Infix, Postfix and Prefix Expressions

- Infix
  - Expressions in which operands surround the operators
  - **Example:** A+B-C

- Postfix or Reverse Polish Notation (RPN)
  - Operators comes after the operands
  - **Example:** AB+C-

- Prefix or Polish Notation
  - Operator comes before the operands
  - **Example:** -+ABC

Example: Conversion From Infix to Postfix

- Infix: A+B*C

- **Conversion:** Applying the rules of precedence
  - A + (B*C) Parentheses for emphasis
  - A + (BC*) Convert the multiplication
  - ABC*+ Postfix Form

Example: Conversion From Infix to Postfix (2)

- Infix: ((A+B)*C-(D-E)) % (F+G)

- Conversion: Applying the rules of precedence

  ((AB+)*C-(DE-)) % (FG+)
  ((AB+C*)-(DE-)) % (FG+)
  (AB+C*DE--) % (FG+)
  AB+C*DE- -FG+%

- Exercise: Convert the following to Postfix

  (A + B) * (C – D)
  A / B * C – D + E / F / (G + H)

Infix, Postfix and Prefix Expressions – Examples

Infix	Postfix	Prefix
A+B	AB+	+AB
(A+B) * (C + D)	AB+CD+*	*+AB+CD
A-B/(C*D^E)	?	?

Why Do We Need Prefix and Postfix?

- Normally, algebraic expressions are written using Infix notation
  - For example: (3 + 4) × 5 – 6

- Appearance may be misleading; Infix notations are not as simple as they seem
  - Operator precedence
  - Associativity property

- Operators have precedence: Parentheses are often required
  - (3 + 4) × 5 – 6 = 29
  - 3 + 4 × 5 – 6 = 17
  - (3 + 4) × (5 – 6) = –7
  - 3 + 4 × (5 – 6) = –1

Why Do We Need Prefix and Postfix?

- Infix Expression is Hard To Parse and difficult to evaluate

- Postfix and prefix do not rely on operator priority and are easier to parse
  - No ambiguity and no brackets are required

- Many compilers first translate algebraic expressions into some form of postfix notation
  - Afterwards translate this postfix expression into machine code

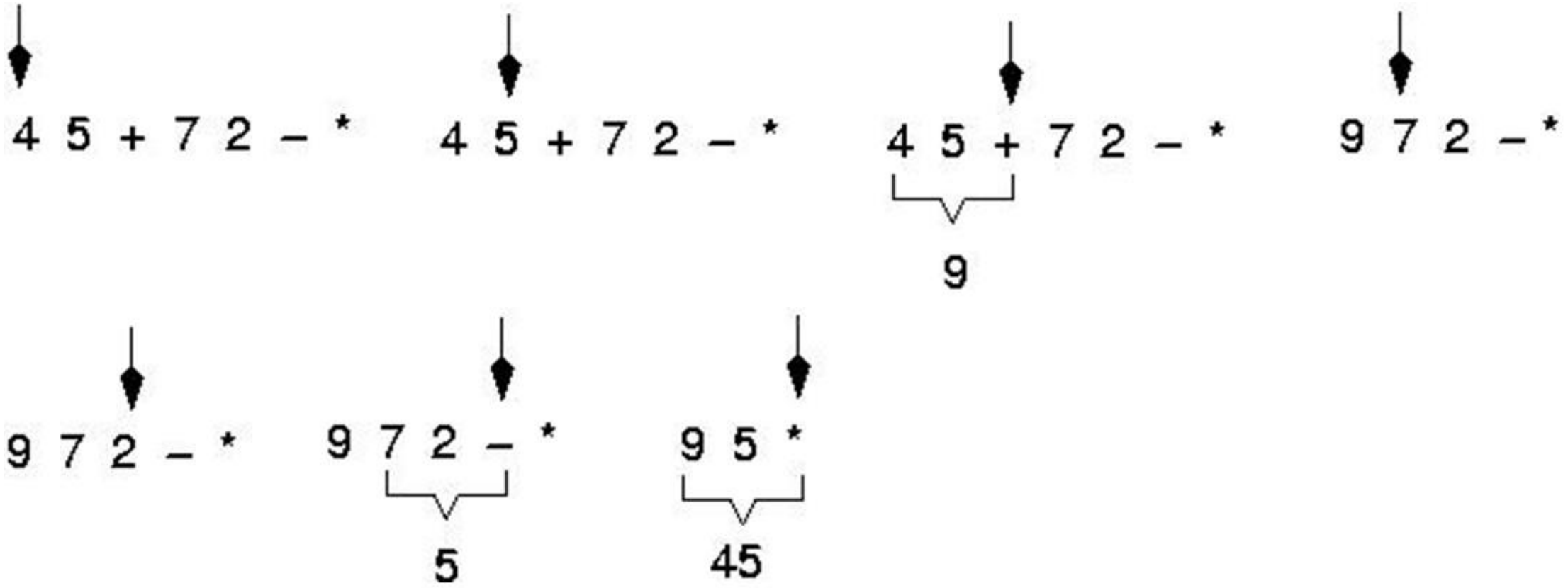
Expression Evaluation (Major Challenges)

- We face two problems to evaluate an expression

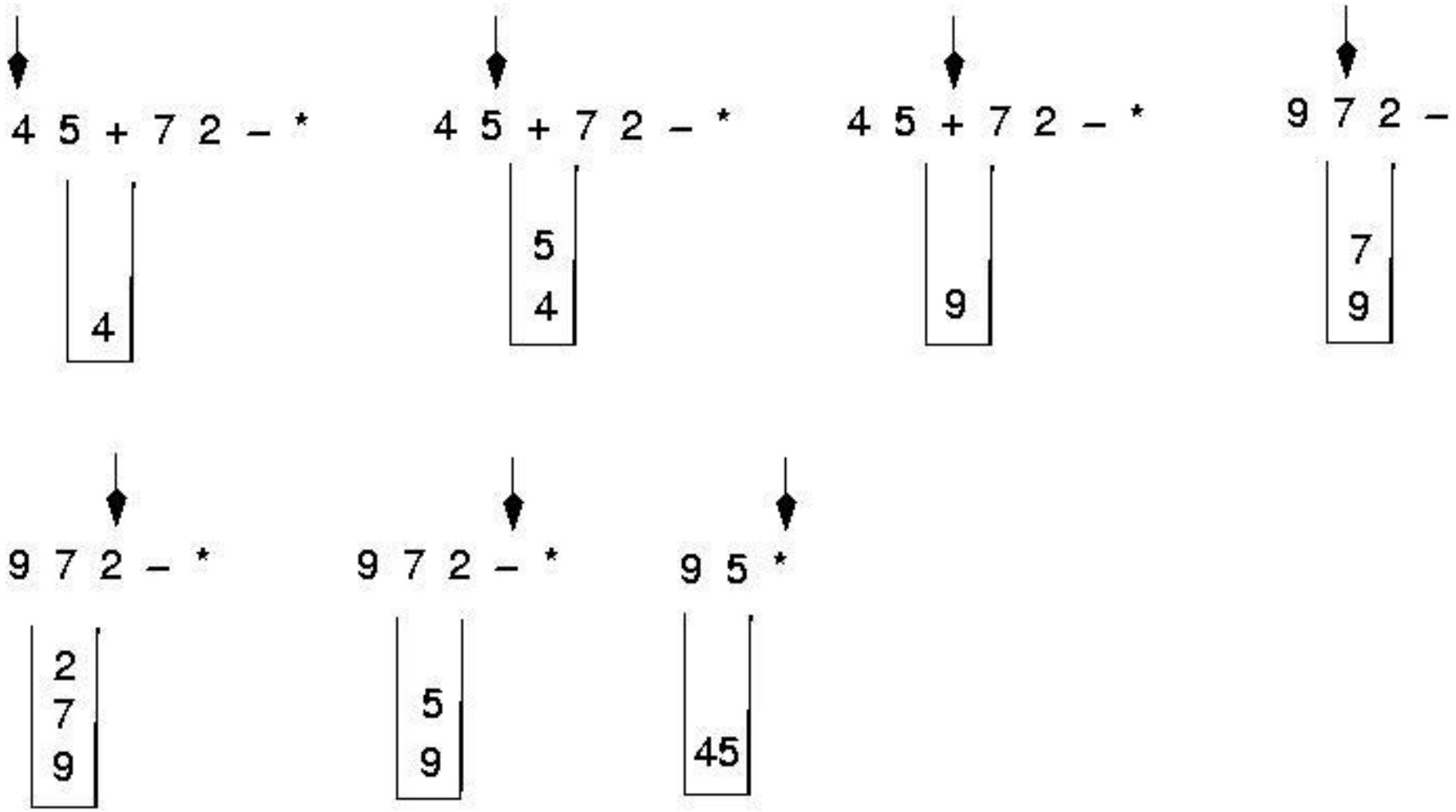
1. Given an infix expression, convert it into a postfix expressions
2. Evaluate a postfix expression

Post Fix Expression Evaluation

Example: Postfix Expressions Evaluation



Example: Postfix Expressions Evaluation and Use of Stack



Example: Postfix Expressions Evaluation and Use of Stack

Quick Exercise

- What does the following postfix expression evaluate to?
  6 3 2 + *

  A. 18
  B. 36
  C. 24
  D. 11
  E. 30

Evaluating a Postfix Expression

```
Let stack be a new Stack object
/* scan the input string reading one element */
/* at a time into symb */
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        stack. push(symb)
    else {
        /* symb is an operator */
          stack. pop(opnd2);
          stack. pop(opnd1);
          result = result of applying symb
                 to opnd1 and opnd2;
        stack. push(result);
    } /* end else */
} /* end while */
stack. pop(final_result); //add final result to final_result
```

Each operator in postfix string refers to the previous two operands in the string.

Evaluating a Postfix Expression

Example Postfix Expression:
6 2 3 + - 3 8 2 / + * 2 $ 3 +

```
Let stack be a new Stack object
/* scan the input string reading one element */
/* at a time into symb */
while (not end of input) {
   symb = next input character;
   if (symb is an operand)
      stack. push(symb)
   else {
      /* symb is an operator */
         stack. pop(opnd2);
         stack. pop(opnd1);
         result = result of applying symb
               to opnd1 and opnd2;
      stack. push(result);
   } /* end else */
} /* end while */
stack. pop(final_result);
```

symb	opnd1	opnd2	result	stack

Evaluating a Postfix Expression

Example Postfix Expression:
6 2 3 + - 3 8 2 / + * 2 $ 3 +

```
Let stack be a new Stack object
/* scan the input string reading one element */
/* at a time into symb */
while (not end of input) {
   symb = next input character;
   if (symb is an operand)
      stack. push(symb)
   else {
      /* symb is an operator */
         stack. pop(opnd2);
         stack. pop(opnd1);
         result = result of applying symb
               to opnd1 and opnd2;
      stack. push(result);
   } /* end else */
} /* end while */
stack. pop(final_result);
```

symb	opnd1	opnd2	result	stack
6				6
2				6,2
3				6,2,3
+	2	3	5	6,5
-	6	5	1	1
3	6	5	1	1,3
8	6	5	1	1,3,8
2	6	5	1	1,3,8,2
/	8	2	4	1,3,4
+	3	4	7	1,7
*	1	7	7	7
2	1	7	7	7,2
$	7	2	49	49
3	7	2	49	49,3
+	49	3	52	52

Infix to Postfix Conversion Using Stack

Conversion of Infix Expression to Postfix – Rules If Expression Does not Contain Parenthesis

- Token is an operand
  - Append it to the end of postfix string
- Token is an operator, *, /, +, or –
  - First remove any operators already on the opstk OR stack that have higher or equal precedence and append them to the postfix string
  - Push the token on the opstk OR stack
- Input expression has been completely processed
  - Any operators still on the opstk OR stack should be removed and appended to the end of the postfix string

- Example: 4 - 2 + 6 * 2

Conversion of Infix Expression to Postfix

- Precedence function
  - `prcd(op1, op2)`
  - `op1` and `op2` are characters representing operators

- Precedence function returns `TRUE`
  - If `op1` has precedence over `op2` **(OR)** `op1 is same as op2`
- Otherwise function returns `FALSE`

- Examples
  - `prcd('*','+')` returns `TRUE`
  - `prcd('+','+')` returns `TRUE`
  - `prcd('+','*')` returns `FALSE`

Algorithm to Convert Infix to Postfix

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A+B*C

symb	Postfix string	opstk
A	A	

Algorithm to Convert Infix to Postfix

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A+B*C

symb	Postfix string	opstk
A	A	
+	A	+

Algorithm to Convert Infix to Postfix

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A+B*C

symb	Postfix string	opstk
A	A	
+	A	+
B	AB	+

Algorithm to Convert Infix to Postfix

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A+B*C

symb	Postfix string	opstk
A	A	
+	A	+
B	AB	+
*	AB	+ *

Algorithm to Convert Infix to Postfix

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A+B*C

symb	Postfix string	opstk
A	A	
+	A	+
B	AB	+
*	AB	+ *
C	ABC	+ *

Algorithm to Convert Infix to Postfix

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A+B*C

symb	Postfix string	opstk
A	A	
+	A	+
B	AB	+
*	AB	+ *
C	ABC	+ *
	ABC*	+

Algorithm to Convert Infix to Postfix

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A+B*C

symb	Postfix string	opstk
A	A	
+	A	+
B	AB	+
*	AB	+ *
C	ABC	+ *
	ABC*	+
	ABC*+	

Algorithm to Convert Infix to Postfix - Exercise

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A * B + C

symb	Postfix string	opstk

Algorithm to Convert Infix to Postfix - Exercise

```
opstk = the empty stack;
while (not end of input) {
    symb = next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while (!empty(opstk) && prcd(stacktop(opstk),symb) ) {
            topsymb = pop(opstk);
            add topsymb to the postfix string;
        } /* end while */
        push(opstk, symb);
    } /* end else */
} /* end while */
/* add remaining operators to string*/
while (!empty(opstk) ) {
    topsymb = pop(opstk);
    add topsymb to the postfix string;
} /* end while */
```

Example: A * B + C

symb	Postfix string	opstk
A	A	
*	A	*
B	AB	*
+	AB*	+
C	AB*C	+
	AB*C+	